

Multi Backup

Manuale utente

Sicuro • Incrementale • Multipiattaforma

RELEASE CANDIDATE 1 | IT | NOVEMBER 2026

Indice

01 Panoramica e primo avvio

02 Installazione e requisiti

03 Dashboard

04 Selezione del plugin

05 Autenticazione GitHub

06 Autenticazione Dropbox

07 Esecuzione e avanzamento

08 Pulizia degli artifact CI

09 Ripresa, riavvio e migrazione

10 Struttura degli archivi cifrati

11 Esplorazione e anteprima

12 Checkout e mount di sola lettura

13 Gestione delle chiavi age

14 Preferenze dell’applicazione

15 Aggiornamenti e canali

16 Esempi CLI

17 Controlli di ripristino

18 Sicurezza e risoluzione dei problemi

19 Sistemi supportati e release

20 Sviluppi successivi e segnalazioni

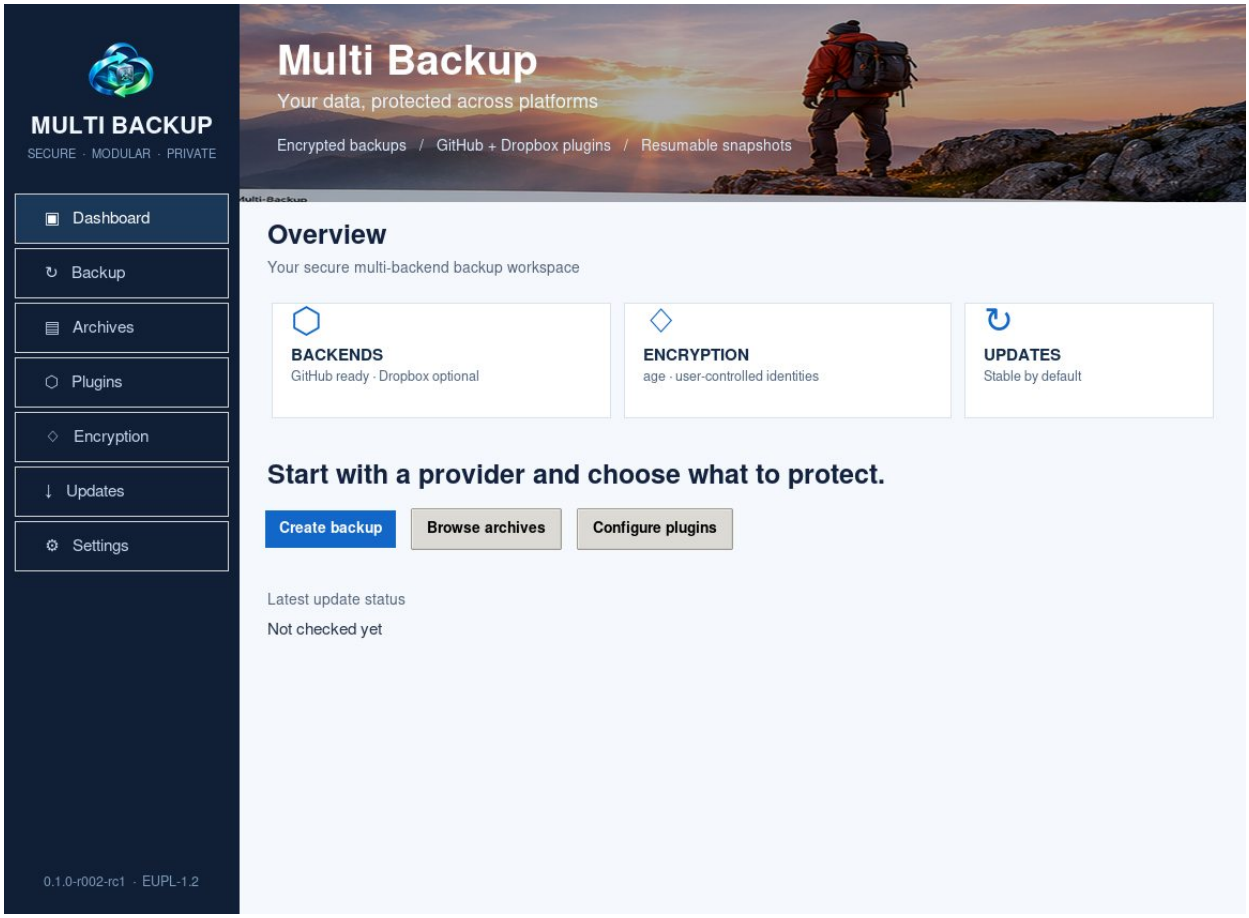
Nota: gli screenshot sono catture reali della GUI RC Linux, non simulazioni di backup effettuati.

01 Panoramica e primo avvio

Multi Backup riunisce motori di backup indipendenti in uno spazio di lavoro cifrato. La prima release candidate mantiene il motore GitHub di disaster recovery e aggiunge il plugin Dropbox incrementale e di sola lettura.

La navigazione comprende Dashboard, Backup, Archivi, Plugin, Cifratura, Aggiornamenti e Impostazioni. Interfaccia grafica e CLI condividono la configurazione dell'applicazione.

- Prima di avviare un lavoro controllare i plugin attivi.
- Impostare un percorso di output riservato su volume cifrato.
- Conservare la chiave privata age separata dal backup.



Dashboard reale catturata su Linux; sui diversi sistemi i controlli possono variare.

02 Installazione e requisiti

Scaricare il pacchetto del proprio sistema operativo: DMG per macOS 13+ (Apple Silicon o Intel), EXE/ZIP per Windows 11 x64, RPM per Rocky/AlmaLinux 8, 9 e 10. I pacchetti RC devono essere verificati e non sono ancora certificati come produzione.

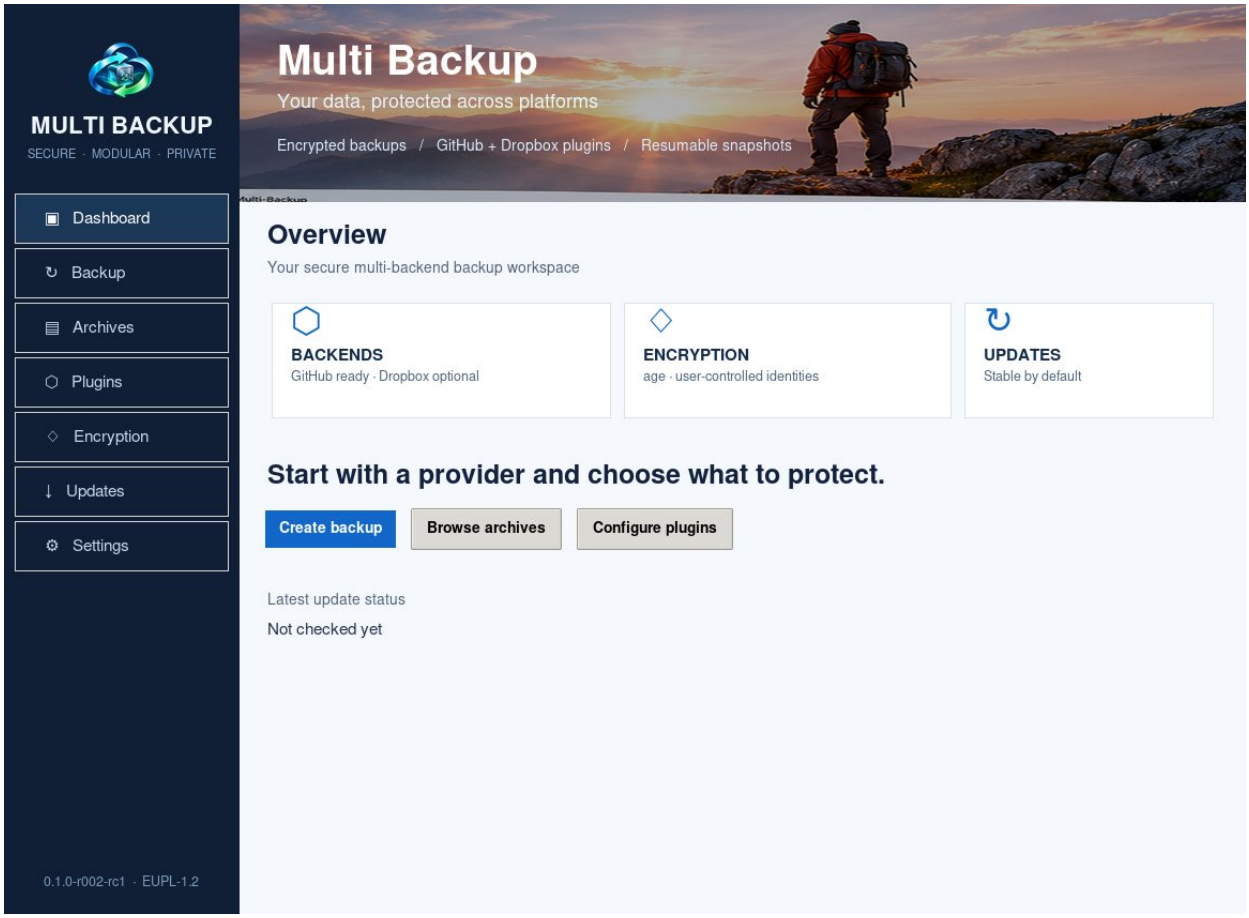
Per eseguire il sorgente servono Python 3.11+, i moduli in requirements.txt e i programmi esterni age, git, Git LFS e GitHub CLI. Il plugin Dropbox richiede age e un token con permessi minimi di lettura.

- Controllare le firme dell’installer e lo SHA-256.
- Verificare che le dipendenze native siano state incluse o installate.
- Usare gh auth login per GitHub; non mostrare i token nelle schermate.

03 Dashboard

La Dashboard fornisce una vista immediata delle sorgenti, del modello di cifratura e del canale di aggiornamento. I pulsanti permettono di creare un backup, consultare gli archivi o aprire la configurazione dei plugin.

Il software non è un servizio di sincronizzazione cloud: il ripristino avviene verso una destinazione selezionata, senza sovrascrivere implicitamente i dati online.



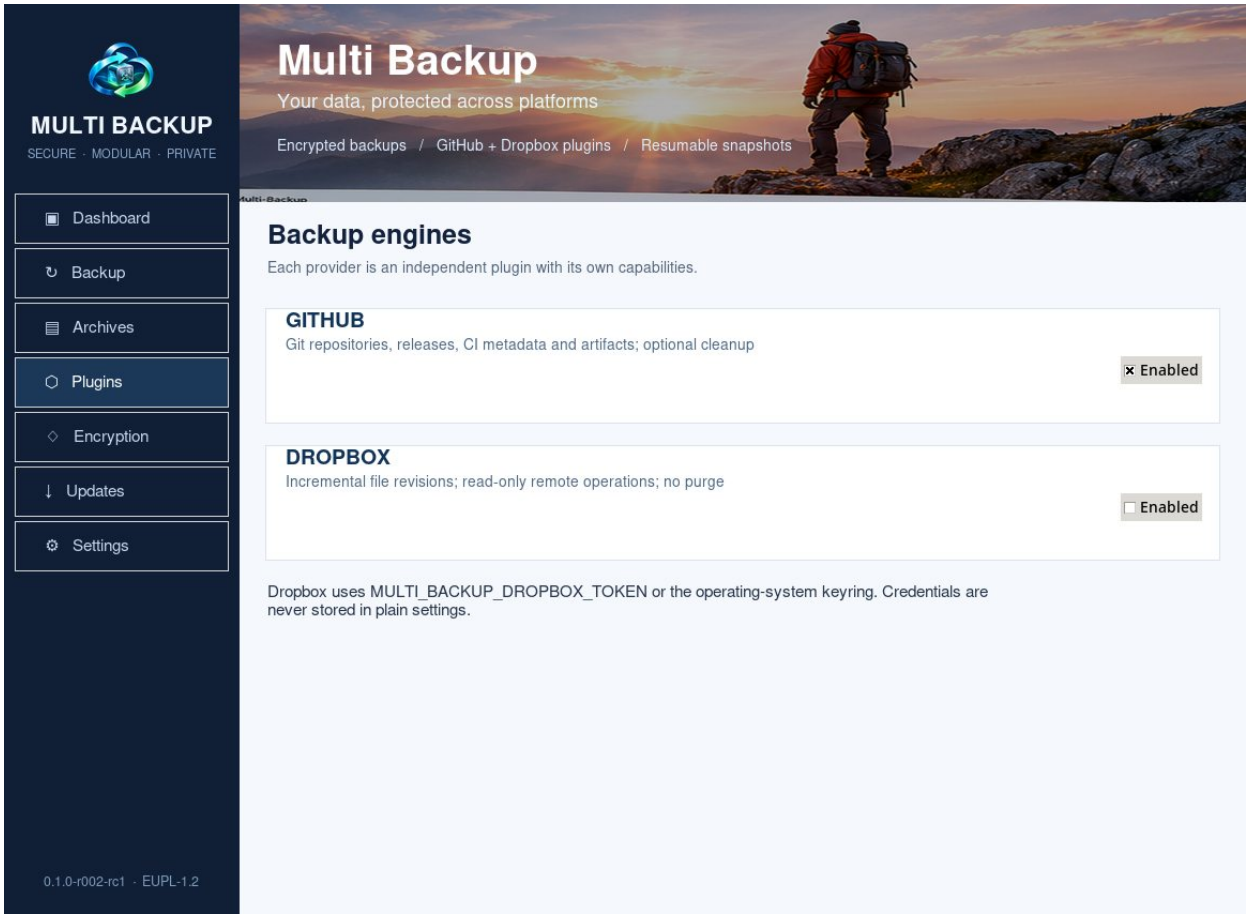
Schermata iniziale con immagine fotografica e scorciatoie.

04 Selezione del plugin

Aprire Plugin per abilitare il motore necessario. GitHub è abilitato per default, mentre Dropbox resta disattivato finché non viene abilitato esplicitamente. Il selettore in Backup determina la sorgente da elaborare.

Il registro interno non carica automaticamente codice Python sconosciuto da cartelle arbitrarie: nella RC sono disponibili soltanto i motori distribuiti con l’applicazione.

- GitHub: repository, release, log CI e pulizia facoltativa degli artifact.
- Dropbox: versioni dei file, backup incrementale e cifratura.
- Dropbox non esegue API remote di delete o purge.



Pagina di gestione dei plugin e del loro stato.

05 Autenticazione GitHub

Autenticarsi con gh auth login e assicurarsi che il token abbia accesso ai repository da conservare. GitHub non permette di leggere i valori originali dei Secrets già registrati.

I segreti originali, se disponibili, possono essere inclusi solo fornendo intenzionalmente una cartella di credenziali su disco cifrato. I registry esterni e i repository di distribuzione possono richiedere politiche di backup proprie.

- Non inserire credenziali in ticket o file README.
- Provare il cleanup soltanto in dry-run, prima della cancellazione.
- Controllare le dipendenze tra CI e repository di distribuzione.

06 Autenticazione Dropbox

Creare un token Dropbox con autorizzazioni di sola lettura. Abilitare Dropbox dai Plugin e usare MULTI_BACKUP_DROPBOX_TOKEN oppure il portachiavi del sistema (servizio multi-backup, utente dropbox).

Il motore legge l’elenco dei file e le revisioni, identifica ogni versione tramite ID e revisione, riutilizza gli archivi già verificati via SHA-256 e produce un manifest cifrato a ogni ciclo.

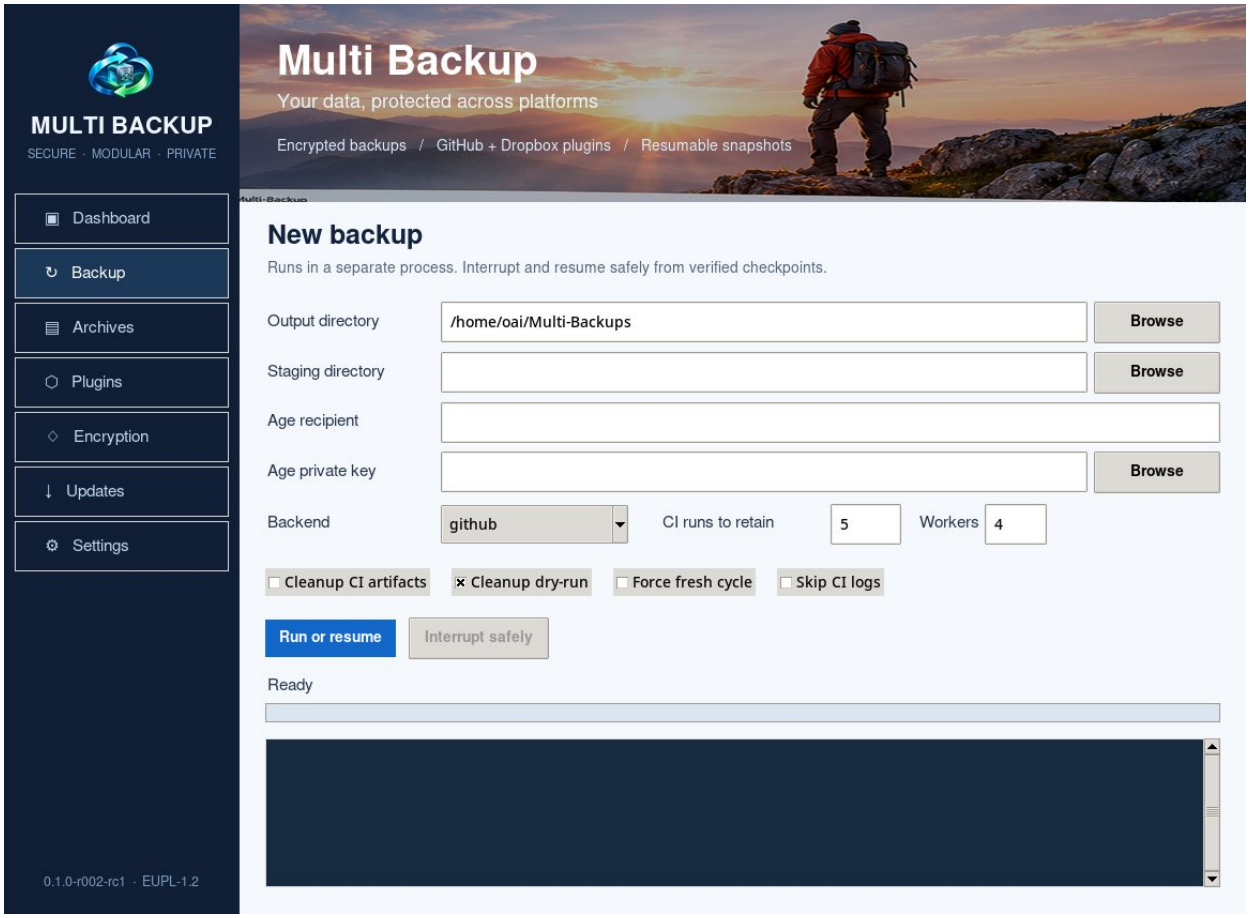
- Non modifica e non cancella file su Dropbox.
- Non effettua purge dei vecchi backup locali.
- Condivisioni, permessi di team e impostazioni amministrative sono fuori scope nella RC.

07 Esecuzione e avanzamento

Nella pagina Backup scegliere backend, cartelle di output/staging e destinatario age. La cartella di staging deve essere privata e su disco cifrato. Il pulsante di avvio lancia un processo separato.

La barra avanza in base agli eventi forniti dalla CLI e l'area dei log mostra attività, warning e sommari. L'interruzione sicura mantiene gli archivi atomici completati e i checkpoint validi.

- Non usare uno staging condiviso o non cifrato.
- Con GitHub ripetere il comando identico per riprendere.
- Usare Force fresh cycle per iniziare un nuovo ciclo indipendente.



Pannello Backup con motore selezionabile, chiavi, simulazione cleanup e barra.

08 Pulizia degli artifact CI

Il cleanup GitHub è facoltativo e precede il backup. Mantiene le ultime CI del branch principale secondo il numero configurato (default cinque), tutela le CI associate ai tag e preserva dinamicamente gli artifact necessari o potenzialmente necessari per la distribuzione.

Eseguire prima Cleanup dry-run: la cancellazione reale è irreversibile. Durante l’operazione vengono mostrati avanzamento per artifact, sommario per repository e riepilogo complessivo. Non vengono eliminati commit, tag o release.

- Attivare espressamente il cleanup.
- Controllare le decisioni conservative prima di applicare.
- Verificare eventuali dipendenze esterne non dichiarate nei workflow.

CLI example / Esempio CLI

```
multi-backup backup --backend github --cleanup-ci-artifacts --cleanup-ci-dry-run --keep-ci-runs 5
```

09 Ripresa, riavvio e migrazione

Il motore GitHub salva per ogni archivio completato un checkpoint con verifica SHA-256. Dopo una interruzione, lo stesso comando riprende dall’ultima unità verificata.

Con --restart si crea un nuovo ciclo senza sovrascrivere gli archivi precedenti. I vecchi metadata monolitici cifrati possono essere ripartiti usando identità age originale e destinatario per i nuovi archivi; la sorgente originale resta immutata fino alla verifica.

- Non cancellare .state prima di una verifica di ripristino.
- Il file in download al momento dell’interruzione potrebbe essere riscaricato.

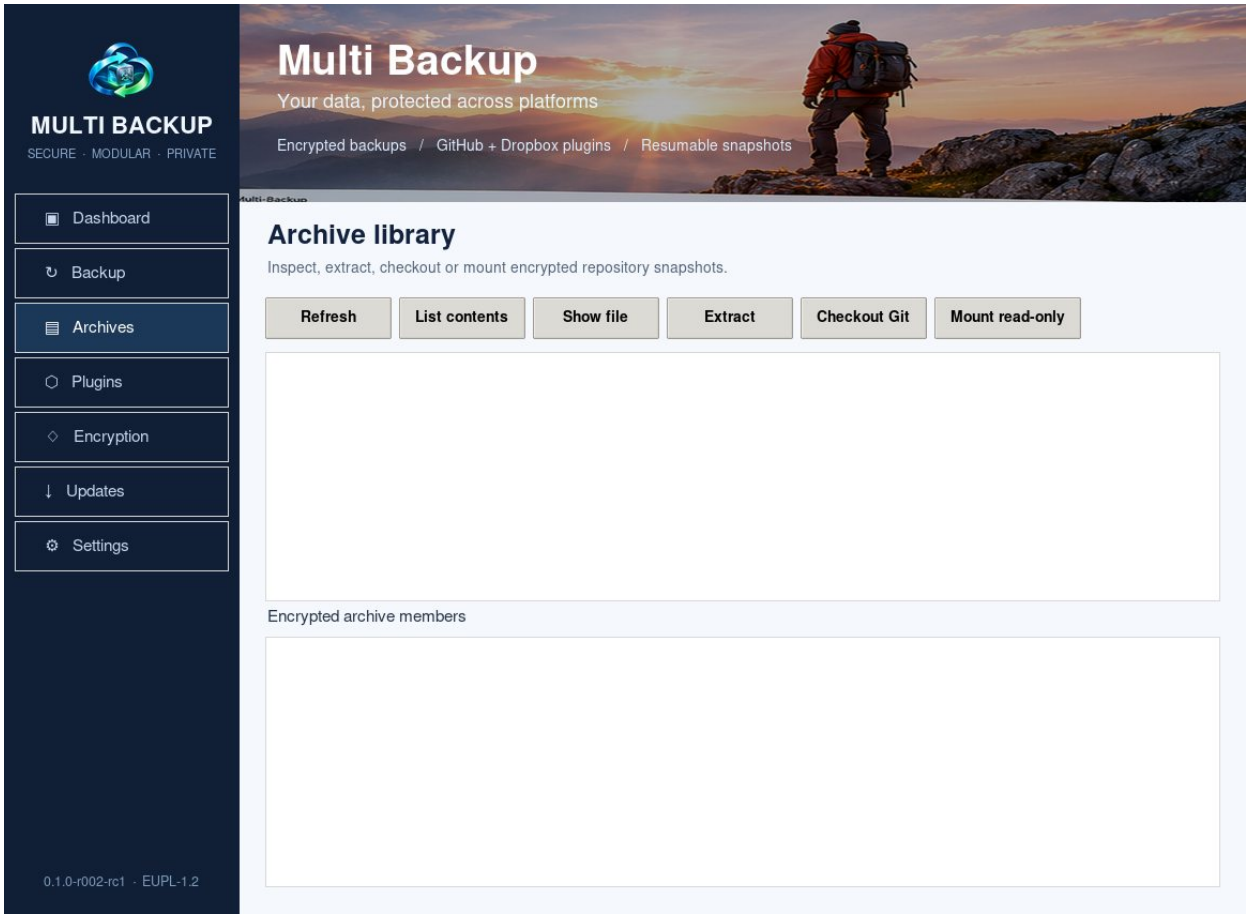
CLI example / Esempio CLI

```
multi-backup backup --backend github --recipient age1ESEMPIO --restart --progress
```

10 Struttura degli archivi cifrati

GitHub conserva account e dati comuni, impostazioni per repository, issue, release e singole esecuzioni CI; i mirror Git usano archivi cifrati separati e possono essere deduplicati.

Dropbox memorizza oggetti cifrati e manifest cifrati per ciclo. Le revisioni non cambiate vengono riutilizzate, senza purge remoto o locale. I file sidecar SHA-256 permettono di verificare la corruzione accidentale.



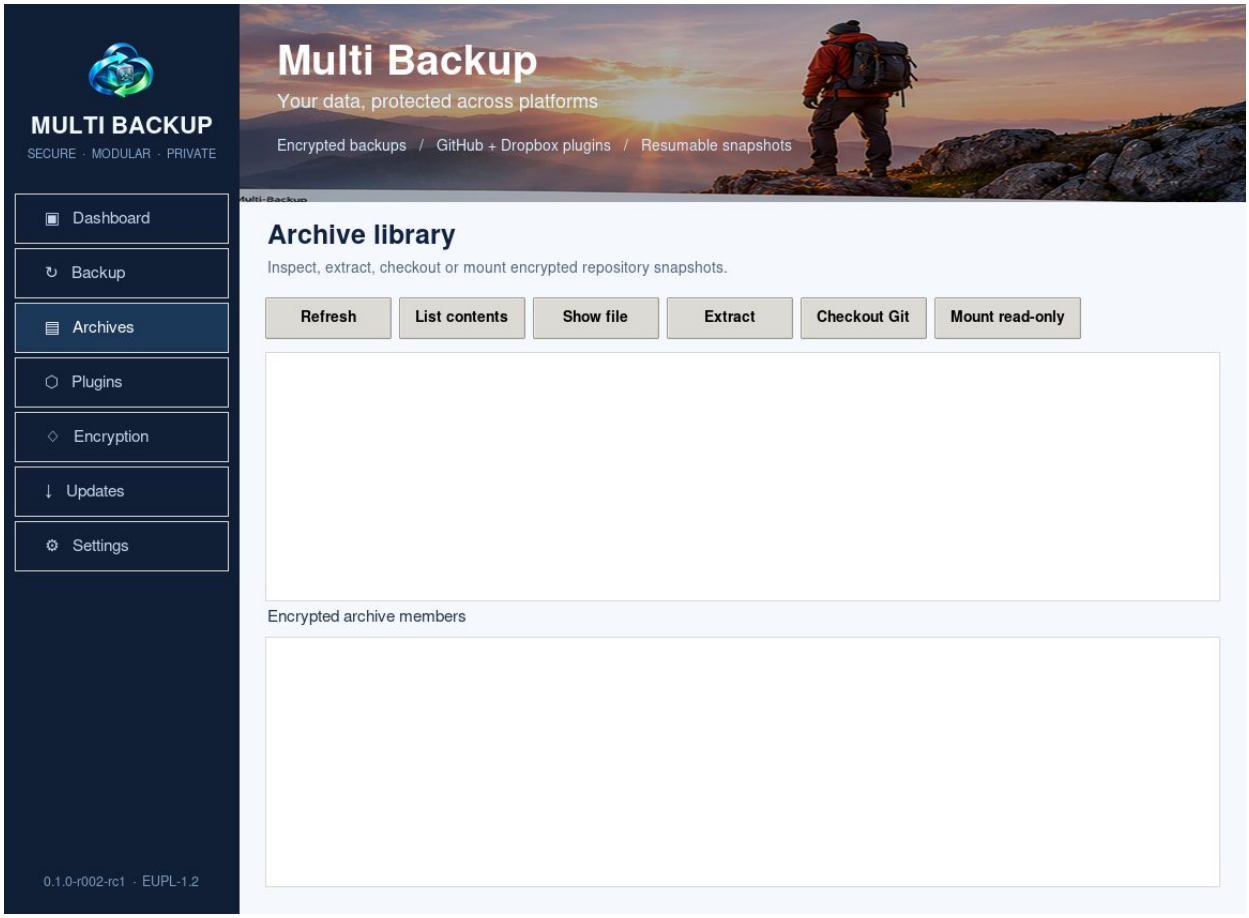
Catalogo dei backup: metadata cifrati e archivi repository.

11 Esplorazione e anteprima

Selezionare un archivio e la chiave privata age associata, quindi scegliere List contents per vedere i file interni. Show file mostra il testo di un membro JSON o di log senza estrarre il contenuto in una cartella utente.

La CLI mette a disposizione list, browse, view, show, extract. L’estrazione verifica i percorsi ed esclude membri pericolosi come link simbolici o attraversamenti di cartelle.

- Non estrarre in cartelle condivise.
- Usare una destinazione nuova con permessi privati.
- I mirror Git richiedono un checkout dedicato.



Comandi della finestra Archivi.

12 Checkout e mount di sola lettura

Checkout Git converte il mirror bare salvato in una working copy presso una destinazione nuova. Mount read-only usa bindfs su Linux/macOS oppure subst su Windows, dopo preparazione di una copia temporanea decifrata.

Lo spazio di lavoro del mount contiene dati in chiaro; usare volume cifrato e smontare al termine. Non è previsto un filesystem cifrato virtuale senza file temporanei.

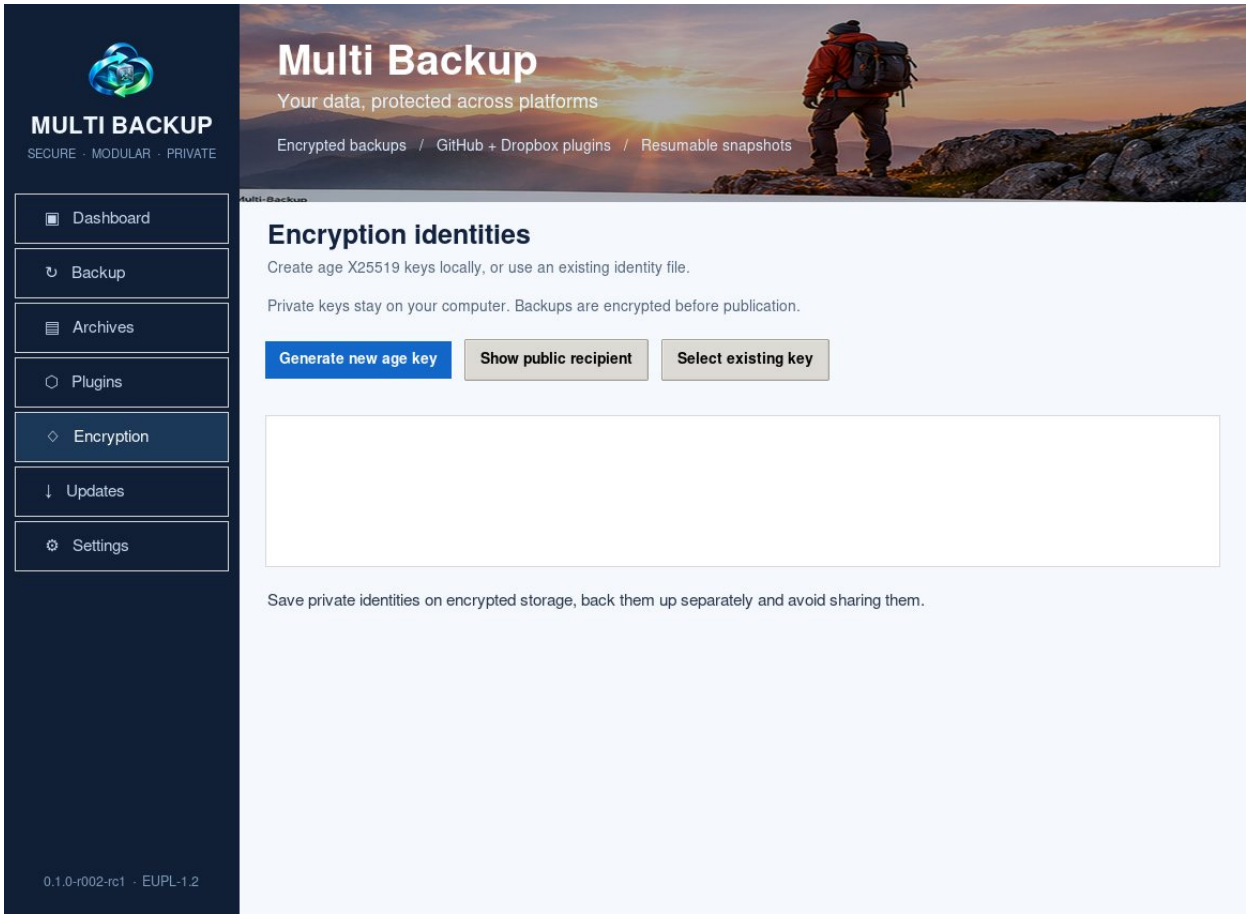
- Installare bindfs dove necessario.
- Concludere con unmount e controllare la pulizia dello staging.

13 Gestione delle chiavi age

Nella pagina Encryption si può generare un’identità X25519. La CLI keys generate crea la chiave nella cartella riservata dell’applicazione, imposta permessi 0600 e rende disponibile la sola chiave pubblica.

Il destinatario pubblico può comparire nelle impostazioni. La chiave privata deve essere conservata fuori dal deposito cifrato, con copie di recupero offline e prove periodiche di decifratura.

- La perdita della chiave privata impedisce di recuperare il backup.
- Non mostrare valori privati negli screenshot.
- Per generare le chiavi può servire il programma age-keygen.



Schermata reale per la gestione delle chiavi.

14 Preferenze dell’applicazione

La pagina Settings mantiene preferenze non segrete: destinazione dei backup, plugin attivi, limiti di conservazione e canale di aggiornamento. Su macOS usa Application Support, su Windows APPDATA e su Linux XDG_CONFIG_HOME o ~/.config.

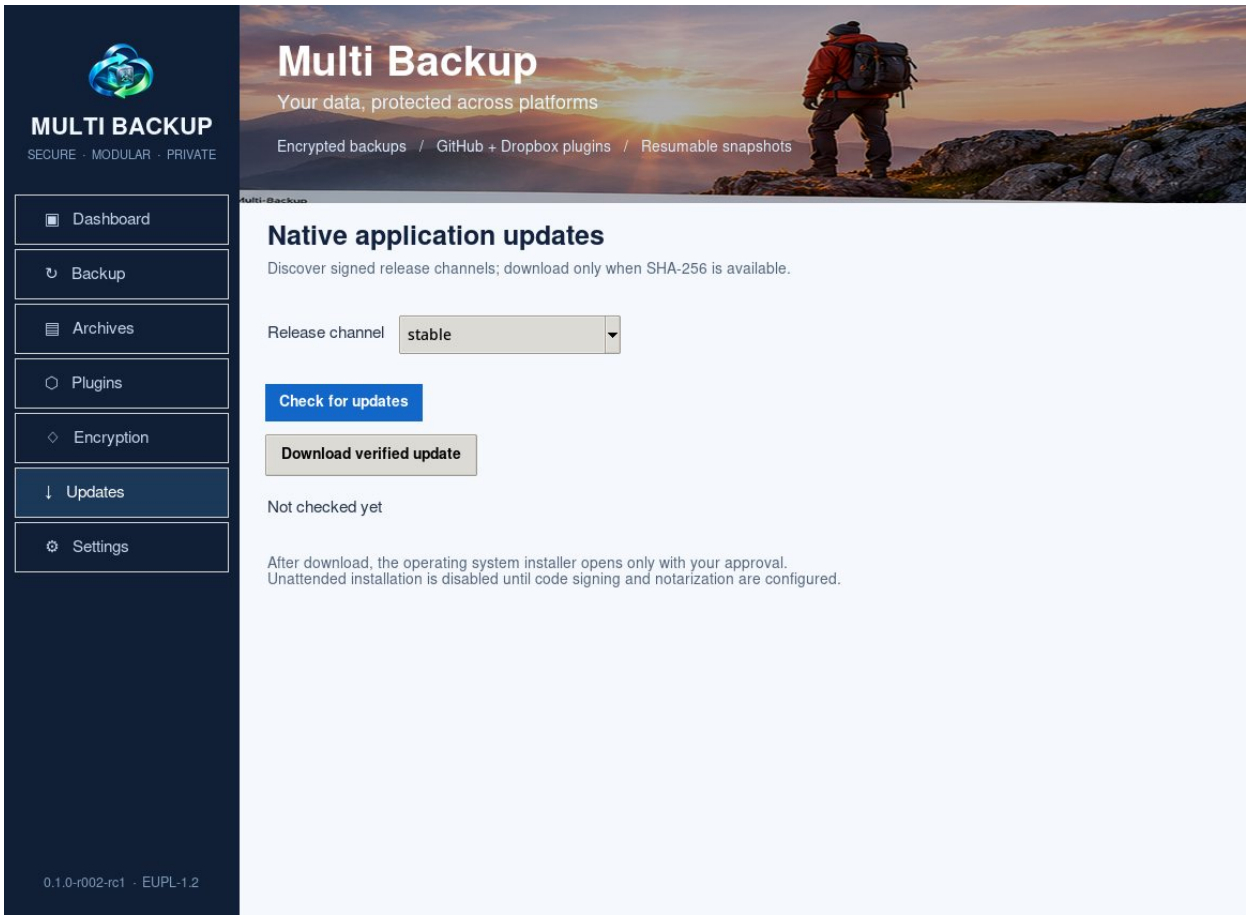
Il file delle impostazioni viene mantenuto privato, dove previsto dal sistema. I token Dropbox non sono salvati nel JSON delle preferenze.

15 Aggiornamenti e canali

Stable è il canale predefinito; Early release è selezionabile nelle impostazioni o nella schermata Updates. Le versioni anticipate vengono prodotte dai commit [full-ci] su main, quelle stabili dai tag di versione dopo il superamento dei test nativi.

La RC controlla i nuovi pacchetti all’avvio e periodicamente (sei ore per default) e scarica soltanto file con hash SHA-256 pubblicato da GitHub. L’installazione senza intervento è bloccata finché mancano firme e notarizzazione.

- Preferire stable per uso quotidiano.
- Verificare sempre la firma del produttore.
- La early release può variare più rapidamente.



Schermata Updates e scelta del canale.

16 Esempi CLI

Tutte le operazioni principali sono disponibili da terminale. Il backend deve essere selezionato per ogni backup; i plugin si possono abilitare dalla CLI. --progress abilita la barra e --json-progress produce eventi integrabili.

L’help dei comandi è esclusivamente in inglese. Gli esempi seguenti coprono versione, plugin, cifratura, cleanup e aggiornamenti.

CLI example / Esempio CLI

```
multi-backup --version
multi-backup plugins
multi-backup plugin enable dropbox
multi-backup keys generate --name primary
multi-backup backup --backend github --progress
multi-backup settings channel early-release
multi-backup updates check
```

17 Controlli di ripristino

Un backup è utile soltanto se è stato verificato un ripristino. Periodicamente elencare gli archivi, verificare i checksum SHA-256, decifrare un piccolo metadata, effettuare checkout di un mirror Git e controllare la cronologia.

Per Dropbox selezionare una revisione e ripristinarla in una directory locale sicura, senza sovrascrivere i dati sorgente. Conservare procedure e identità di emergenza fuori dai backup.

- Conservare la chiave age offline e separata.
- Provare ripristini su un host pulito.
- Controllare i sommari di backup incompleti e cleanup CI.

18 Sicurezza e risoluzione dei problemi

Se un backup non riesce, esaminare il log e i checkpoint prima di ripetere. Credenziali gh mancanti, age non disponibile, permessi dello staging e token scaduti sono cause comuni. I checkpoint impediscono di sovrascrivere gli archivi completati validi.

I metadata GitHub non comprendono i valori originali dei Secrets né tutti i sistemi esterni. I pacchetti RC richiedono test d’installazione, firma e revisione di sicurezza prima della produzione.

- Errore 401/403: controllare credenziali e permessi.
- Archivio non leggibile: verificare checksum e identità.
- Metadata storici: fornire identità age originale e destinatario nuovo.

19 Sistemi supportati e release

Il progetto punta a macOS Ventura 13+ su Apple Silicon/Intel, Windows 11 x64, RockyLinux/AlmaLinux 8/9/10 x86_64. La CI usa runner o container corrispondenti. Tutti i job richiesti devono concludersi correttamente prima della pubblicazione.

Il tag rolling early-release raccoglie i pacchetti anticipati, mentre i tag v0.1.0 e successivi identificano le stable. Le definizioni repository RPM restano disabilitate in attesa della firma GPG.

20 Sviluppi successivi e segnalazioni

La prima release candidate è un traguardo di verifica, non una certificazione production. I passaggi successivi sono test nativi degli installer, migrazione reale di archivi cifrati, prove Dropbox e validazione end-to-end di installazione/aggiornamento/ripristino.

Per segnalare problemi usare le issue del repository desalvo/multi-backup, indicando versione, sistema operativo, parametri CLI senza credenziali, log anonimizzati e passi di riproduzione. L’handoff si trova in docs/HANDOFF.md.